

Day3 · 22차시

개선 루프 & 스케일

작동하는 앱을, 더 좋게 그리고 더 크게.
관찰한 것을 개선으로, 작동하는 것을 규모로.

SECTION 00

오프닝 — first run이 끝이 아니다

한 번 작동시키는 것 vs 항상 싸게·빠르게·안정적으로 작동시키는 것

“한 번 작동시키는 것”과 “항상 안정적으로, 싸게, 빠르게 작동시키는 것” — 무엇이 다른가?

— 에이전트의 진짜 난제는 first run이 아니라 지속 운영

지난 차시에서 여기까지

▶ 관찰은 끝이 아니라 시작 — 로그는 다음 개선의 연료

18차시 — 관찰
트레이스·로그로
무슨 일이 일어나는지



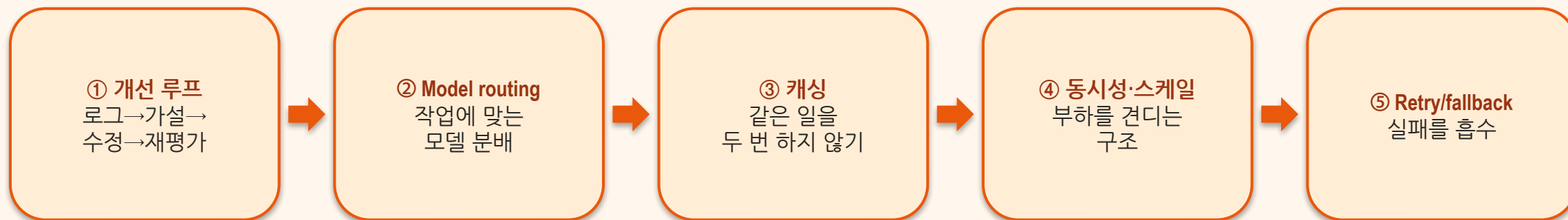
21차시 — 안전
가드레일·승인으로
사고를 막았다



22차시(오늘)
그 데이터로 개선하고
작동하는 것을 확장

이 차시 동안 얻는 것

▶ 결국 '측정하고, 고치고, 아낀다'는 하나의 이야기



운영에 들어가면 보이는 것

▶ 품질·비용·안정성은 운영하면서 드러난다 — 그래서 개선과 스케일이 한 세트

반복

같은 질문이 계속 —
캐시로 회피 가능

10x

쉬운 요청에 최상위 모델
= 과지출

실패

API는 언젠가 죽는다 —
흡수 설계 필요

SECTION 01

개선 루프 — 로그에서 개선으로

한 번 좋다고 계속 좋지 않다

▶ '완성'이 아니라 '계속 좋아지는 시스템'을 만든다

확률성

모델은 확률적 — 같은 입력에도 답이 흔들린다

세상의 변화

입력·사용자·세상이 바뀌면 품질이 떨어진다 → **distribution shift**

결론

일회성 튜닝이 아니라 **반복되는 개선 사이클**

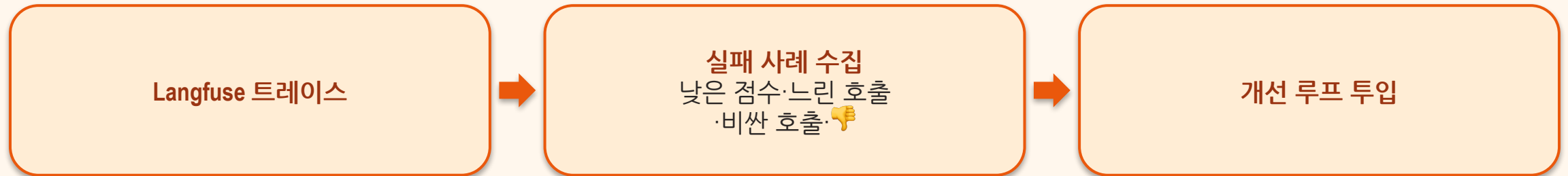
관점 전환

출시는 끝이 아니라 시작점

입력 = 트레이스

트레이스가 개선의 연료다

▶ 관찰성이 없으면 고칠 대상을 모른다



어디서 개선 신호를 얻나

▶ 셋을 합쳐야 전체 그림 — 명시는 정확하지만 드물고, 암묵은 풍부하지만 해석이 필요

명시적 피드백
사용자의 직접 평가
👍/👎 · 별점 · 신고

암묵적 피드백
행동 신호 —
재질의·이탈·복불 여부

프로덕션 트레이스
실행 기록 그 자체 —
도구 호출·경로·실패

피드백 파이프라인 — 원시 데이터에서 액션까지

▶ 자동화가 규모를, 사람이 판단을 — 두 층이 겹쳐야 한다

원시 텔레메트리

로그·트레이스·지표 — 프로덕션에서 쏟아지는 그대로

자동 분석층

이상 탐지·규칙 트리거·이슈 군집화(clustering)

사람 검토층

군집을 보고 판단·검증·우선순위

실행 가능한 제안

프롬프트 수정·도구 개선·데이터 보강

개선 1사이클의 4단계

▶ 한 번에 한 변수만 바꿔야 원인을 안다

1

로그 수집

트레이스에서 실패·저품질 사례를 모은다 (평가셋이 된다)

2

가설

왜 틀렸나 — 프롬프트? 도구? 모델? 원인을 추정

3

수정

한 군데만 바꾼다 — 프롬프트 한 줄·도구 설명·모델 교체

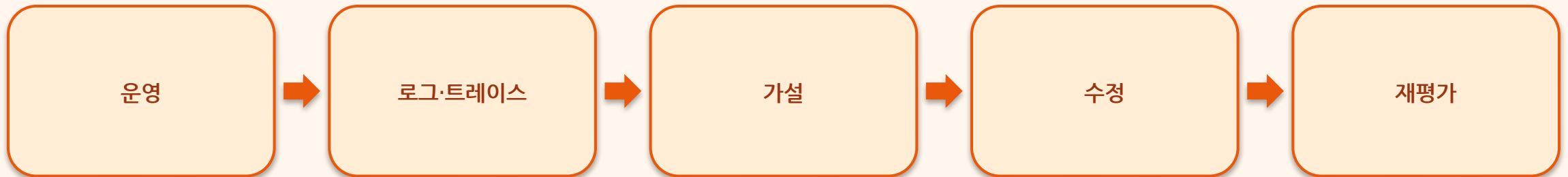
4

재평가

같은 평가셋으로 다시 채점 — 정말 좋아졌나 숫자로 확인

개선 루프 — 닫힌 고리

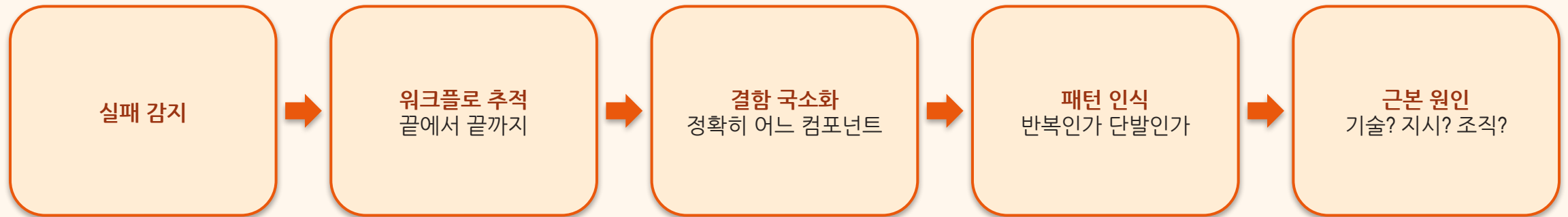
▶ 열린 루프(고치고 끝)는 문제를 못 잡는다 — 닫아야 학습이 쌓인다



🔄 좋아졌으면 배포, 아니면 다시 — 숫자로 확인하고 돌아온다

RCA(Root Cause Analysis) — 실패의 뿌리까지 내려가라

▶ 흔한 뿌리: 드리프트 · 모호한 지시 · 도구 파라미터 오류 · 스킬 선택 불일치



프롬프트 개선 vs 파인튜닝

▶ 순서는 항상 같다 — 싸고 빠른 것부터

프롬프트·컨텍스트 개선

- 빠르고 싸고 해석 가능
- 드리프트 대응에 최적
- few-shot·도구 설명·검증 보강
- 대부분은 여기서 끝난다

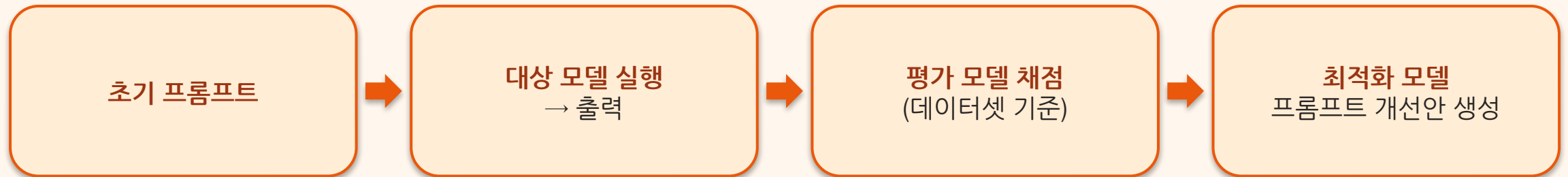
VS

파인튜닝 (LoRA 등)

- 근본적·반복적 패턴에만
- 데이터·비용·시간 부담
- 프롬프트로 안 되는 것이 증명된 후
- 마지막 수단

프롬프트 개선도 자동으로 (APO · DSPy)

▶ 사람 개입 없이 — 최적화 모델이 프롬프트를 다듬는 루프



🔄 개선된 프롬프트로 다시 — 점수가 수렴할 때까지 (DSPy·APO)

수정을 어떻게 배포하나

▶ 앞서 만든 평가셋이 게이트 역할을 한다

즉시 전체 배포

- 바뀐 프롬프트를 한 번에 전원에
- 나빠지면 전 사용자가 피해
- 빠르지만 위험

VS

점진 실험

- **Shadow**: 실서비스 옆에서 몰래 평가
- **A/B**: 일부 트래픽에만
- **Gating**: 점수 미달이면 자동 차단

언제 개선을 멈추나

▶ 개선도 비용이다 — 수확 체감 지점을 숫자로 본다

수확 체감

같은 노력에 개선폭이 계속 줄면 — 멈출 신호

파레토 (80/20)

중요한 20% 이슈가 80% 영향 — 나머지는 기술 부채로 기록만

위험 점수 에스컬레이션

불확실성 × 결과 심각도 — 높은 것만 사람 검토로

판단 기준

다음 개선의 기대 효과 < 비용 이면 stop

SECTION 02

Model routing — 작업에 맞는 모델

한 모델로 다 하면 비싸다

▶ 결론: 요청마다 맞는 모델을 골라 보내자 = 라우팅

단순 질문에 최상위 모델

- “2+2는?”에 추론 모델
- 돈·시간 낭비
- 대부분의 트래픽이 단순

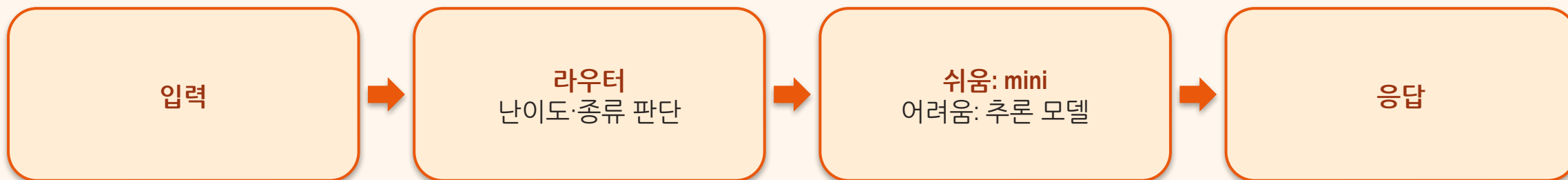
VS

복잡 질문에 싼 모델

- 다단계 추론에 mini
- 품질 실패
- 틀린 답이 더 비싸다

Model routing이란

▶ 라우터 자체는 가볍고 빨라야 — 배보다 배꼽이 커지면 안 된다



라우팅 기준은 어떻게

▶ 대부분 트래픽은 싼 모델이 처리, 소수의 어려운 것만 비싼 모델로

1

휴리스틱 — 여기서 시작

입력 길이·키워드('분석'·'비교'·'왜')로 난이도 추정

2

분류 모델

작은 모델이 난이도·카테고리를 먼저 분류

3

에스컬레이션

싼 모델 먼저 → 자신 없으면 강한 모델로 재시도

SECTION 03

캐싱 — 같은 일을 두 번 하지 마라

프롬프트 캐시 vs 시맨틱 캐시

▶ 실무에서는 최종 응답만이 아니라 임베딩·검색 결과까지 총총이 캐싱

프롬프트 캐시 (정확 일치)

- 동일 입력의 결과 재사용
- 해시(key) → 응답
- 빠르고 단순·오적중 없음
- 한 글자만 달라도 미스

VS

시맨틱 캐시 (의미 유사)

- 비슷한 질문에도 적중
- 임베딩 유사도로 매칭
- 적중률↑
- 오적중 위험 — 임계값 필요

캐시가 주는 것

▶ 응답을 빠르게 만드는 게 아니라 — 일 자체를 없앤다

캐시 MISS

- 모델을 실제로 호출
- 느림 (수백 ms~초)
- 토큰 비용 발생

VS

캐시 HIT

- 저장된 답 즉시 반환
- 거의 0 ms
- 모델 호출 0 → 무료

캐시가 틀릴 때

▶ 무지성 캐싱은 오히려 사고를 만든다 — 무엇을 캐시할지 선별

신선도 (stale)

데이터가 바뀌면 캐시가 옛 답을 준다 → TTL·무효화 필요

시맨틱 오적중

비슷해 보여도 다른 질문 → 유사도 임계값 튜닝

부적합 대상

개인화·실시간 응답은 캐시 부적합

측정

'캐시 적중률'도 측정 대상 — 히트율과 정확도의 균형

SECTION 04

규모와 견고함 — 부하·실패를 견디기

규모 확장은 한 방향이 아니다

▶ 어느 차원이 병목인지 먼저 측정

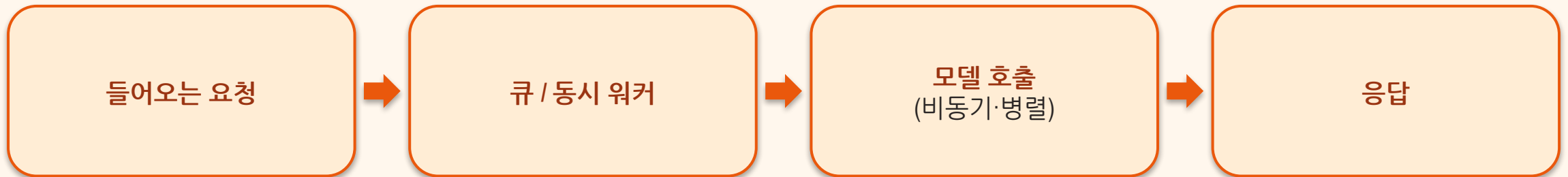
① 데이터 양
문서·임베딩이 늘면
저장·검색 인덱스가 병목

② 데이터 복잡도
문서가 길고 복잡해지면
처리·청킹 비용↑

③ 쿼리 부하 (QPS)
동시 요청이 늘면
지연·타임아웃

부하를 견디는 구조

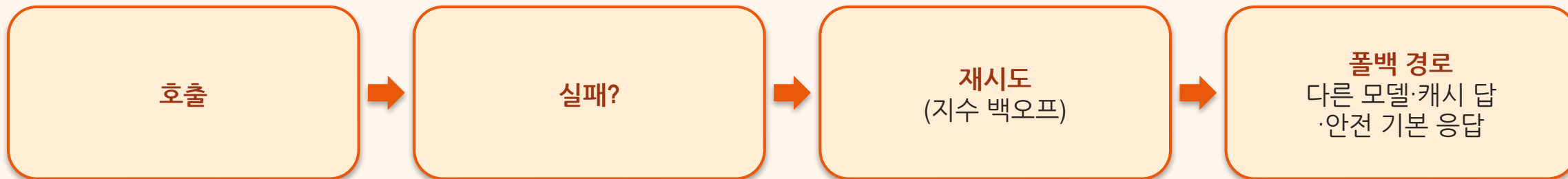
▶ 품질용 컴포넌트(리랭킹 등)를 더하면 지연이 다시 증가한다 — 처리량과 지연의 균형



🔗 QPS가 늘면 수평 확장 · rate limit · 비동기 호출로 저지연 유지

Retry / fallback — 실패를 흡수

▶ 무한 재시도 금지(상한 필수) — 폴백의 목적은 사용자가 빈 화면을 안 보게



🔗 한도 초과까지 재시도 → 그래도 안 되면 폴백으로 응답

비용을 통제하라

▶ 진짜 큰 비용 = 프롬프트 컨텍스트 포함 LLM 입력 토큰 — 절감 두 축 = 캐싱 + 라우팅

저장 비용
벡터DB·문서 저장
양에 비례

컴퓨터 비용
임베딩·검색·추론 —
LLM 입력 토큰이 최대

제3자 API 비용
모델·외부 서비스
호출 누적

SECTION 05

정리

오늘 3줄

- 개선 = 로그→가설→수정→재평가의 닫힌 루프 — 트레이스(18차시)가 연료, 멈출 줄도 알기
- 스케일 = 라우팅·캐싱·동시성·retry/fallback로 싸고·빠르고·견고하게
- 절감도 개선도 측정 없이는 불가능 — 점수·적중률·지연·비용을 항상 같이 본다

측정하지 않으면 개선할 수 없고, 개선하지 않으면 비용만 늘어난다.

— 관찰 → 개선 → 규모, 그리고 다시 관찰